

MERN / MEAN Stack

Complete Interview Answer Guide

Accion Labs — All Rounds Covered

JavaScript · React · Angular · Node.js · MongoDB · Express · System Design

130 Detailed Answers with Real-World JavaScript Code Examples

Rounds 2 · 3 · 4 · 5 · Bonus Topics

2026 Edition

ROUND 2 — JavaScript Core Concepts (Q34–Q50)

Q34. What are closures and give a real-world use case?

A closure is a function that remembers the variables from its outer (lexical) scope even after that outer function has returned. In JavaScript, every function forms a closure over the environment in which it was created. This is not a bug or an edge-case — it is a fundamental language feature that enables data privacy, factory functions, memoization, and event handlers.

How it works: When a function is created, the JS engine does not copy the outer variables. Instead it keeps a live reference to the outer scope's variable environment. Even after the outer function finishes, the inner function still holds that reference alive.

Real-world Example 1 — Counter with private state:

```
function createCounter() {
  let count = 0;          // private – cannot be touched from outside

  return {
    increment() { count++; },
    decrement() { count--; },
    value()     { return count; }
  };
}

const counter = createCounter();
counter.increment();
counter.increment();
console.log(counter.value()); // 2
console.log(count);          // ReferenceError – count is private
```

Real-world Example 2 — Event handler factory (button IDs):

```
function makeHandler(buttonId) {
  return function() {
    console.log(`Button ${buttonId} clicked`); // closes over buttonId
  };
}

document.getElementById('btn1').addEventListener('click', makeHandler('btn1'));
document.getElementById('btn2').addEventListener('click', makeHandler('btn2'));
```

Real-world Example 3 — Memoization:

```
function memoize(fn) {
  const cache = {}; // closed over by the returned function
  return function(...args) {
    const key = JSON.stringify(args);
    if (cache[key] !== undefined) return cache[key];
    cache[key] = fn(...args);
    return cache[key];
  };
}

const expensiveSquare = memoize(n => {
  console.log('computing...');
  return n * n;
});

expensiveSquare(5); // computing... 25
expensiveSquare(5); // 25 (served from cache – no log)
```

Why it matters: Closures are the backbone of React's `useState` hook, Redux store, Node.js middleware chains, and virtually every library you use daily.

Q35. Explain hoisting — how does it work for `var`, `let`, `const`, and functions?

Hoisting is JavaScript's behaviour of moving declarations to the top of their scope during the compilation phase, before any code executes. Only the declaration is hoisted — not the initialisation.

var: Declaration is hoisted to the top of the function scope and automatically initialised to undefined. You can read it before its line but get undefined.

let / const: Declarations ARE hoisted to the top of their block scope, but they are NOT initialised. Accessing them before their line throws a `ReferenceError` — this window is called the Temporal Dead Zone (TDZ).

Function declarations: Fully hoisted — both the declaration AND the body. You can call a function declaration before it appears in the source code.

Function expressions / arrow functions: Only the variable declaration is hoisted (as `var/let/const`). The function body is NOT hoisted.

```
// --- var ---
console.log(a); // undefined (hoisted, not initialised)
var a = 10;
console.log(a); // 10

// --- let / const (TDZ) ---
// console.log(b); // ReferenceError: Cannot access 'b' before init
let b = 20;

// --- Function Declaration (fully hoisted) ---
greet(); // "Hello!" - works before the declaration
function greet() { console.log("Hello!"); }

// --- Function Expression (NOT hoisted) ---
// sayHi(); // TypeError: sayHi is not a function
var sayHi = function() { console.log("Hi!"); };

// --- Real-world pitfall with var in a loop ---
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100); // prints 3, 3, 3 (NOT 0,1,2)
}

// Fix: use let instead
for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log(j), 100); // prints 0, 1, 2
}
```

Q36. What is the event loop? How does it handle async operations?

JavaScript is single-threaded — it has one call stack. Yet it handles async operations (network calls, timers, I/O) without blocking. This magic is achieved through the event loop, which coordinates the call stack with several queues.

Components: Call Stack (LIFO — where executing code lives), Web APIs / Node APIs (where async work actually happens — browser or libuv), Microtask Queue (Promises, `queueMicrotask`, `MutationObserver` — highest priority), Macrotask Queue / Task Queue (`setTimeout`, `setInterval`, I/O callbacks — lower priority).

Event loop cycle: 1) Execute everything currently on the call stack. 2) Drain the entire microtask queue (run ALL pending microtasks). 3) Take ONE macrotask from the task queue and execute it. 4) Drain microtasks again. 5) Repeat.

```
console.log('1 - synchronous');

setTimeout(() => console.log('2 - macrotask (setTimeout)'), 0);

Promise.resolve().then(() => console.log('3 - microtask (Promise)'));

queueMicrotask(() => console.log('4 - microtask (queueMicrotask)'));

console.log('5 - synchronous');

// Output order:
// 1 - synchronous
// 5 - synchronous
// 3 - microtask (Promise)
// 4 - microtask (queueMicrotask)
// 2 - macrotask (setTimeout)
```

Real-world impact — why this matters in Node.js API servers:

```
// Blocking the event loop starves ALL other requests:
app.get('/bad', (req, res) => {
  // Heavy sync computation – blocks the loop for ALL users
  const result = heavySyncComputation(); // BAD
  res.json(result);
});

// Correct: offload to async or worker threads
app.get('/good', async (req, res) => {
  const result = await computeInWorker(); // non-blocking
  res.json(result);
});
```

Q37. Difference between == and ===

=== (strict equality) checks both value AND type — no conversion is performed. == (loose equality) performs type coercion before comparing: it converts one or both operands to a common type using complex rules defined in the ECMAScript spec.

In practice: always use `===` unless you have a specific reason for type coercion. The rules for `==` are notoriously counterintuitive and a common source of bugs.

```
// === never coerces
console.log(1 === '1'); // false (number vs string)
console.log(0 === false); // false (number vs boolean)
console.log(null === undefined); // false

// == coerces in surprising ways
console.log(1 == '1'); // true ('1' coerced to 1)
console.log(0 == false); // true (false coerced to 0)
console.log(null == undefined); // true (special rule)
console.log(null == 0); // false (null only == undefined)
console.log('' == false); // true (both coerced to 0)
console.log([] == false); // true ([] -> '' -> 0, false -> 0)

// Real-world tip: API data from JSON is always strings
const userId = '42'; // came from query param or JSON
userId === 42; // false - catches the bug
userId == 42; // true - silently hides it
```

Q38. What are Promises? How do they differ from callbacks?

A Promise is an object representing the eventual completion or failure of an asynchronous operation. It has three states: pending (initial), fulfilled (resolved with a value), or rejected (failed with a reason). Promises were introduced to solve callback hell — deeply nested callbacks that are hard to read, reason about, and handle errors in.

Key differences from callbacks: 1) Promises are composable — you can chain `.then()/.catch()`. 2) Error propagation is automatic — a rejection travels down the chain until caught. 3) Promises are eager — they start executing immediately on creation. 4) Promises guarantee single invocation — a callback can be called multiple times.

[illegible]

Q40. What is the difference between call, apply, and bind?

All three methods allow you to explicitly set what 'this' refers to inside a function. The difference is in how they are invoked and how arguments are passed.

call: Invokes the function immediately. Arguments passed individually after the context.

`apply`: Invokes the function immediately. Arguments passed as an array (useful when args are already in an array).

bind: Does NOT invoke the function. Returns a NEW function with 'this' permanently bound. Used for event handlers and partial application.

[illegible]

Q41. Explain prototypal inheritance in JavaScript

In JavaScript, every object has an internal link to another object called its prototype. When you access a property or method that doesn't exist on an object, the engine walks up the prototype chain until it finds it or reaches null. This is prototype-based inheritance — different from class-based inheritance in Java/C++.

Every function has a `.prototype` property. When you use `'new'`, a new object is created with its `[[Prototype]]` set to the constructor's `.prototype`. ES6 class syntax is just syntactic sugar over this same mechanism.

[illegible]

[illegible]

Q43. What is the Temporal Dead Zone (TDZ)?

The Temporal Dead Zone is the period between the start of a block scope and the point where a `let` or `const` declaration is initialised. During the TDZ, the variable exists (is hoisted) but is not accessible. Any attempt to read or write it throws a `ReferenceError`.

This is actually a safety feature. With `var` you silently get undefined before initialisation, which hides bugs. With `let/const` you get an immediate, explicit error.

[illegible]

Q44. What is the 'this' keyword? How does its value change in different contexts?

'this' refers to the execution context — the object that is currently running the code. Its value is determined at runtime based on HOW a function is called, not where it is defined (except for arrow functions which use lexical scope).

```
// 1. Global context
console.log(this); // window (browser) or {} (Node.js module)

// 2. Regular function call (strict mode – undefined; sloppy – global)
function show() { console.log(this); }
show(); // undefined (strict) or window (sloppy)

// 3. Method call – 'this' = the object before the dot
const user = {
  name: 'Alice',
  getName() { return this.name; }
};
user.getName(); // 'Alice'

// 4. Losing context
const fn = user.getName;
fn(); // undefined – 'this' is no longer 'user'

// 5. new – 'this' = newly created object
function Person(name) { this.name = name; }
const p = new Person('Bob'); // this.name = 'Bob'

// 6. call / apply / bind – explicit 'this'
user.getName.call({ name: 'Charlie' }); // 'Charlie'
```



```
// 7. Arrow function – lexical 'this' from enclosing scope
const obj = {
  value: 100,
  getArrow: function() {
    const inner = () => this.value; // captures 'this' from getArrow
    return inner();
  }
};
obj.getArrow(); // 100

// 8. Event handler – 'this' = the DOM element
button.addEventListener('click', function() {
  console.log(this); // the button element
});
// With arrow: 'this' is the enclosing scope (e.g., class instance)
```

Q45. Difference between null and undefined

undefined means a variable has been declared but has not been assigned a value yet. It is JavaScript's own way of saying 'no value'. null is an explicit assignment — a developer intentionally sets a variable to null to say 'this has no value' or 'this intentionally empty'.

```
let a;
console.log(a);           // undefined – declared but not assigned

let b = null;
console.log(b);           // null – intentionally empty

// typeof
console.log(typeof undefined); // 'undefined'
console.log(typeof null);      // 'object' – historical JS bug

// Equality
console.log(null == undefined); // true (loose – special rule)
console.log(null === undefined); // false (strict – different types)

// Real-world: API responses
const user = fetchUser(id);
if (user === null) {
  // User was looked up but doesn't exist in the DB
}
if (user === undefined) {
  // The property was never set – maybe a code bug
}

// Optional chaining with null/undefined
const city = user?.address?.city ?? 'Unknown';
// ?? (nullish coalescing) returns right side only for null/undefined
// (unlike || which also triggers for 0, '', false)
```

Q46. What is event bubbling and event capturing?

When an event occurs on a DOM element, it goes through three phases: Capture phase (event travels from the root down to the target), Target phase (event reaches the target element), Bubble phase (event travels back up from target to the root). By default, addEventListener listens in the bubble phase. To listen in the capture phase, pass {capture: true} as the third argument.

```
// HTML structure: document > div#outer > div#inner > button
const outer = document.getElementById('outer');
const inner = document.getElementById('inner');
const button = document.getElementById('btn');

// Bubbling (default) – inner fires first, then outer
inner.addEventListener('click', () => console.log('inner bubble'));
outer.addEventListener('click', () => console.log('outer bubble'));
// Click button: "inner bubble", then "outer bubble"

// Capturing – outer fires first
outer.addEventListener('click', () => console.log('outer capture'), true);
inner.addEventListener('click', () => console.log('inner capture'), true);
// Click button: "outer capture", "inner capture", "inner bubble", "outer bubble"
```

```
// stopPropagation – prevent further bubbling/capturing
button.addEventListener('click', (e) => {
    e.stopPropagation(); // stops the event from bubbling up
    console.log('button clicked');
});

// ■■■ Event Delegation (practical use of bubbling) ■■■■■■■■■■■■■■
// Instead of adding listeners to 100 list items, add ONE to the parent
const list = document.getElementById('todo-list');
list.addEventListener('click', (e) => {
    if (e.target.matches('li')) {
        console.log('Clicked:', e.target.textContent);
        e.target.classList.toggle('done');
    }
});
// Works even for items added dynamically AFTER the listener was attached
```

Q47. What are generators in JavaScript?

A generator is a special function that can pause its execution and resume it later. Defined with function*, it uses the yield keyword to pause and optionally produce a value. When called, it returns a Generator object (an iterator). The generator does not execute immediately — it only runs when you call .next() on the iterator.

Generators are powerful for: lazy evaluation of infinite sequences, implementing iterators, async control flow (basis of `async/await` in older transpilers), and cancellable async operations.

[illegible]

Q48. Explain the difference between shallow copy and deep copy

A shallow copy creates a new object but only copies the top-level properties. Nested objects and arrays still share references with the original. A deep copy recursively copies all nested objects/arrays so the copy is completely independent of the original.

```
// ■■■ Shallow copy methods ■■■  
const original = { a: 1, nested: { b: 2 }, arr: [1, 2] };  
  
const s1 = Object.assign({}, original);  
const s2 = { ...original };      // spread operator  
  
s2.a = 99;                       // does NOT affect original  
s2.nested.b = 99;                // DOES affect original – shared reference!  
console.log(original.nested.b); // 99  
  
// ■■ Deep copy options ■■■  
  
// Option 1: JSON (simple but has limitations)  
const d1 = JSON.parse(JSON.stringify(original));  
// Loses: undefined, functions, Date objects, Infinity, circular refs  
  
// Option 2: structuredClone (modern – Node 17+, modern browsers)  
const d2 = structuredClone(original); // handles Date, Map, Set, etc.  
d2.nested.b = 999;  
console.log(original.nested.b); // 2 – unchanged  
  
// Option 3: Lodash (for older environments)  
import _ from 'lodash';  
const d3 = _.cloneDeep(original);  
  
// ■■ Custom recursive deep clone ■■■  
function deepClone(obj) {  
  if (obj === null || typeof obj !== 'object') return obj;  
  if (Array.isArray(obj)) return obj.map(deepClone);  
  return Object.fromEntries(  
    Object.entries(obj).map(([k, v]) => [k, deepClone(v)])  
  );  
}  
  
// ■■ Real-world: Redux state updates (always shallow or deep copy) ■■  
const newState = { ...state, user: { ...state.user, name: 'Bob' } };
```

Q49. What is a WeakMap and when would you use it?

A WeakMap is a collection of key-value pairs where keys must be objects (not primitives), and the keys are held weakly — meaning they do not prevent garbage collection. If the key object is no longer referenced elsewhere, it can be garbage collected and its WeakMap entry is automatically removed. WeakMaps are not iterable, have no size property, and have no way to list all entries.

Use WeakMap when you want to associate metadata with objects without leaking memory — especially when you do not control the object's lifetime.

```
// ■■ Problem WeakMap solves ████████████████████████████████  
const cache = new Map(); // regular Map holds strong ref to DOM nodes  
function process(node) {  
    if (cache.has(node)) return cache.get(node);  
    const result = heavyComputation(node);  
    cache.set(node, result);  
    return result;  
}  
  
// If node is removed from DOM, Map still holds it – MEMORY LEAK  
  
// ■■ With WeakMap – no memory leak ████████████████████████████████  
const weakCache = new WeakMap();  
function processWeak(node) {  
    if (weakCache.has(node)) return weakCache.get(node);  
    const result = heavyComputation(node);  
    weakCache.set(node, result); // auto-cleaned when node is GC'd  
    return result;  
}
```


[illegible]

ROUND 2 — React.js Core Concepts (Q51–Q70)

Q51. What is the Virtual DOM and how does React use it?

The Virtual DOM (VDOM) is a lightweight, in-memory JavaScript representation of the real DOM tree. Instead of manipulating the real DOM directly on every state change (which is slow because DOM operations trigger layout, paint, and composite in the browser), React keeps a virtual copy, computes what changed, and then makes minimal, surgical updates to the real DOM.

Process: 1) On state/prop change, React re-renders the component into a new VDOM tree. 2) React diffs the new tree against the previous VDOM (reconciliation). 3) React computes the minimum set of changes (patch). 4) React applies those changes to the real DOM in a batch.

[illegible]

Q52. Explain the React reconciliation algorithm (React Fiber)

Reconciliation is the process React uses to determine what changed in the VDOM and what needs to be updated in the real DOM. React Fiber (introduced in React 16) is the complete rewrite of the reconciliation engine that makes it incremental and interruptible.

Key heuristics: 1) Different element types produce completely different trees — React destroys the old tree and builds new. 2) Keys help React match list items across renders — without keys React uses index which causes bugs. 3) React processes one Fiber node at a time and can pause to handle higher-priority work (user input).

[illegible]

```
// ■■ shouldComponentUpdate / React.memo ████████████████████████████████████  
// Tell React: "skip reconciliation if props haven't changed"  
const MemoizedList = React.memo(function List({ items }) {  
    return <ul>{items.map(i => <li key={i.id}>{i.text}</li>)}</ul>;  
}, (prevProps, nextProps) => {  
    // return true to skip re-render  
    return prevProps.items === nextProps.items;  
});  
  
// ■■ Fiber priorities ████████████████████████████████████████████████████████  
// React 18 concurrent mode batches and prioritizes work:  
import { startTransition } from 'react';  
  
startTransition(() => {  
    setSearchResults(filterLargeList(query)); // low priority – can be interrupted  
});  
// Meanwhile, typing in the input remains responsive (high priority)
```

Q53. What are React Hooks? List the most commonly used ones

Hooks are functions that let functional components 'hook into' React state and lifecycle features without writing class components. Introduced in React 16.8. Rules of Hooks: only call at the top level (not inside loops/conditions), only call from React functions.

[illegible]

```

        .then(setData)
        .catch(setError)
        .finally(() => setLoading(false));
    }, [url]);

    return { data, loading, error };
}

// Usage:
function App() {
    const { data, loading } = useFetch('/api/users');
    if (loading) return <Spinner />;
    return <UserList users={data} />;
}

```

Q54. Difference between useEffect and useLayoutEffect

Both run after render, but at different times in the browser's pipeline. `useEffect` runs asynchronously AFTER the browser has painted the screen — it does not block the visual update. `useLayoutEffect` runs synchronously AFTER DOM mutations but BEFORE the browser paints — it blocks paint until it finishes.

Use `useLayoutEffect` when you need to measure DOM elements or make DOM mutations that should be invisible to the user (to avoid visual flickers). Use `useEffect` (the default) for everything else — data fetching, subscriptions, logging.

[illegible]

Q55. What is useRef and when do you use it?

useRef returns a mutable object { current: initialValue } that persists for the full lifetime of the component. Two main use cases: 1) Accessing DOM elements directly (like document.getElementById but the React way). 2) Storing any mutable value that should NOT trigger a re-render when changed (unlike useState).

```
import { useRef, useEffect, useState } from 'react';  
  
// ■■ Use case 1: DOM reference  
function SearchBox() {  
  const inputRef = useRef(null);  
  
  useEffect(() => {  
    inputRef.current.focus(); // focus on mount  
  }, []);  
}
```



```
}  
  
// ■■ useCallback – stable function reference ████████████████████  
const Button = memo(function Button({ onClick, label }) {  
    console.log('Button rendered');  
    return <button onClick={onClick}>{label}</button>;  
});  
  
function Parent() {  
    const [count, setCount] = useState(0);  
    const [theme, setTheme] = useState('light');  
  
    // Without useCallback – new function on every render, Button re-renders  
    // const handleClick = () => setCount(c => c + 1);  
  
    // With useCallback – same function reference unless deps change  
    const handleClick = useCallback(() => {  
        setCount(c => c + 1);  
    }, []); // no deps – never recreated  
  
    return (  
        <div>  
            <Button onClick={handleClick} label="Increment" />  
            <button onClick={() => setTheme(t => t === 'light' ? 'dark' : 'light')}>  
                Toggle theme (won't re-render Button above)  
            </button>  
        </div>  
    );  
}
```

Q57. What is Context API and when would you use it over Redux?

Context API is React's built-in way to share data across the component tree without passing props at every level (avoiding prop drilling). Redux is an external state management library with a more structured approach: a single store, actions, reducers, and middleware for side effects.

Use Context API for: theme, locale, auth user, feature flags — data that is read frequently but changes infrequently. Use Redux (or Zustand/Jotai) for: complex state with many transitions, shared state that many components read/write, need for middleware/DevTools/time-travel debugging, or when Context would cause excessive re-renders.

```
// Context API - auth example
import { createContext, useContext, useState } from 'react';

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);

  const login = async (creds) => {
    const u = await api.login(creds);
    setUser(u);
  };

  const logout = () => setUser(null);

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

// Custom hook for consuming context
export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth must be inside AuthProvider');
  return ctx;
}

// Any component anywhere in the tree:
```

[illegible]

Q58. What is the difference between controlled and uncontrolled components?

A controlled component is one where React state drives the form element's value. Every keystroke updates state, and the input always shows what state says. An uncontrolled component stores its own value in the DOM — you access it using a ref. Controlled components give you full control (instant validation, conditional submit) at the cost of more boilerplate.

```
// ■■ Controlled component ████████████████████████████████████████████  
function ControlledForm() {  
    const [name, setName] = useState('');  
    const [email, setEmail] = useState('');  
  
    const handleSubmit = (e) => {  
        e.preventDefault();  
        // name and email always in sync with state  
        api.submit({ name, email });  
    };  
  
    return (  
        <form onSubmit={handleSubmit}>  
            <input  
                value={name}  
                onChange={e => setName(e.target.value)}  
            />  
            <input  
                value={email}  
                onChange={e => setEmail(e.target.value)}  
            />  
            <button>Submit</button>  
        </form>  
    );  
}  
  
// ■■ Uncontrolled component ████████████████████████████████████████████  
function UncontrolledForm() {  
    const nameRef = useRef();  
    const emailRef = useRef();  
  
    const handleSubmit = (e) => {  
        e.preventDefault();  
        // Read value from DOM at submit time  
        api.submit({ name: nameRef.current.value, email: emailRef.current.value });  
    };  
  
    return (  
        <form onSubmit={handleSubmit}>  
            <input ref={nameRef} defaultValue="" />  
            <input ref={emailRef} defaultValue="" />  
            <button>Submit</button>  
        </form>  
    );  
}
```

```
// Use uncontrolled for: file inputs (always uncontrolled), integrating
// with non-React libraries, simple forms where you only need value on submit
```

Q59. Explain React lifecycle methods (class components)

React class components go through three phases: Mounting (component added to DOM), Updating (re-render from state/prop changes), and Unmounting (component removed). Hooks are the functional equivalent — `useEffect` covers most lifecycle needs.

[illegible]

```
return user ? <div>{user.name}</div> : <Spinner />;
}
```

Q60. What are Pure Components and React.memo?

React.PureComponent (class) and React.memo (functional) are optimisation tools that prevent unnecessary re-renders by doing a shallow comparison of props (and state for PureComponent). If nothing changed, they skip the render.

[illegible]

Q61. How do you implement code splitting in React?

Q62. What is lazy loading in React?

Code splitting means breaking the JavaScript bundle into smaller chunks that are loaded on demand rather than all at once. This reduces the initial bundle size and speeds up the first load. `React.lazy` and `Suspense` are the built-in tools for this.

[illegible]

withRouter).

[illegible]

Q65. What is prop drilling and how do you avoid it?

Prop drilling occurs when you pass data through multiple intermediate components that don't actually need the data — they just pass it down to their children. This makes the code hard to maintain because changing the shape of the data requires updating every component in the chain. Solutions: Context API, state management (Redux/Zustand), component composition.

[illegible]

```

return (
  <UserContext.Provider value={user}>
    <Layout2 />      {/* no user prop needed */}
  </UserContext.Provider>
);
}
function Layout2() { return <Sidebar2 />; } // no prop
function Sidebar2() { return <UserAvatar2 />; } // no prop
function UserAvatar2() {
  const user = useContext(UserContext); // gets it directly
  return <img src={user.avatar} alt={user.name} />;
}

// ■■■ Solution 2: Component composition (render props/children) ■
function App3() {
  const [user] = useState({ name: 'Alice' });
  return (
    <Layout>
      <Sidebar>
        <UserAvatar user={user} />  {/* passed as children */}
      </Sidebar>
    </Layout>
  );
}
// Layout and Sidebar just render {children} – no prop drilling

```

Q66. How does React Router work? BrowserRouter vs HashRouter?

React Router is a declarative routing library for React. It maps URL paths to components. It uses the browser History API (or hash) to manage navigation without full-page reloads.

BrowserRouter: Uses HTML5 History API (pushState). URLs look clean: /dashboard. Requires server to serve index.html for all routes (otherwise direct navigation to /dashboard returns 404). BrowserRouter is preferred.

HashRouter: Uses URL hash: /#/dashboard. The server only sees / — everything after # is client-side. Works without server configuration. Useful for static file hosts or when you can't configure server routing.

```

import { BrowserRouter, Routes, Route, Link,
  Navigate, useParams, useNavigate, Outlet } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/users">Users</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/users" element={<UserList />} />
        <Route path="/users/:id" element={<UserDetail />} />

        {/* Protected route */}
        <Route path="/admin" element={
          <RequireAuth><AdminPanel /></RequireAuth>
        } />

        {/* Nested routes */}
        <Route path="/dashboard" element={<DashboardLayout />>
          <Route index element={<DashboardHome />} />
          <Route path="analytics" element={<Analytics />} />
          <Route path="settings" element={<Settings />} />
        </Route>

        <Route path="*" element={<Navigate to="/" replace />} />
      </Routes>
    </BrowserRouter>
  );
}

function UserDetail() {

```



```
const { id } = useParams(); // read :id from URL
const navigate = useNavigate();
return (
  <div>
    <h1>User {id}</h1>
    <button onClick={() => navigate(-1)}>Back</button>
  </div>
);
}

function DashboardLayout() {
  return <div><Sidebar /><Outlet /></div>; // Outlet renders child route
}
```

Q67. What is Server-Side Rendering (SSR) in React? How does Next.js implement it?

SSR means rendering the React component tree to HTML on the server for each request, sending that HTML to the browser. The browser shows content immediately (better FCP/LCP metrics), and then React hydrates (attaches event handlers) on the client. Benefits: better SEO (search bots see full HTML), faster perceived load, better performance on low-end devices.

[illegible]

Q68. Explain the Flux architecture and how Redux implements it

Flux is an application architecture pattern (not a library) introduced by Facebook to manage complex state flows. It enforces unidirectional data flow: Actions -> Dispatcher -> Store -> View. Redux is the most popular implementation of Flux, simplifying the dispatcher into pure reducer functions.

Redux principles: Single source of truth (one store), state is read-only (only actions can change it), changes are made with pure reducer functions (predictable, testable).

[illegible]

Q69. What is Redux Thunk vs Redux Saga?

Both are Redux middleware for handling side effects (async operations like API calls). Redux Thunk is simple: it lets you dispatch functions (thunks) instead of plain objects — the thunk can do async work and dispatch multiple actions. Redux Saga uses ES6 generators to describe complex async flows in a more testable, declarative way.

Use Thunk for: simple async operations, small-medium apps. Use Saga for: complex orchestration (parallel calls, cancellation, debouncing, retry logic, race conditions), when you need highly testable side effects.

```
// ■■■ Redux Thunk ■■■■
// A thunk is just a function that returns a function:
const loginUser = (credentials) => async (dispatch, getState) => {
  dispatch({ type: 'auth/loginStart' });
  try {
    const user = await api.login(credentials);
    dispatch({ type: 'auth/loginSuccess', payload: user });
    dispatch(fetchUserProfile(user.id)); // dispatch another action
  } catch (error) {
```

[illegible]

Q70. How do you handle forms in React? (controlled, Formik, React Hook Form)

There are three main approaches: controlled components (pure React state), Formik (form library with validation), and React Hook Form (performance-focused, uncontrolled internally with a clean API). React Hook Form is currently the most popular choice due to its minimal re-renders and simple API.

[illegible]

```
// ■■■ 2. React Hook Form (recommended) ■■■■  
import { useForm } from 'react-hook-form';  
import { zodResolver } from '@hookform/resolvers/zod';  
import { z } from 'zod';  
  
const schema = z.object({  
  email:      z.string().email('Invalid email'),  
  password:   z.string().min(6, 'Min 6 chars'),  
});  
  
function LoginFormRHF() {  
  const { register, handleSubmit, formState: { errors } } = useForm({  
    resolver: zodResolver(schema)  
  });  
  
  const onSubmit = (data) => api.login(data);  
  
  return (  
    <form onSubmit={handleSubmit(onSubmit)}>  
      <input {...register('email')} />  
      {errors.email && <span>{errors.email.message}</span>}  
      <input type="password" {...register('password')} />  
      {errors.password && <span>{errors.password.message}</span>}  
      <button>Login</button>  
    </form>  
  );  
}  
  
// Advantages: minimal re-renders, built-in validation, easy schema integration
```

ROUND 2 — Angular Core Concepts (Q71–Q80)

Q71. What is two-way data binding in Angular? How does ngModel work?

Two-way data binding means the view (template) and the component's model (TypeScript class) stay in sync automatically. When the user changes input in the view, the model updates. When the model changes, the view updates. Angular achieves this with the `[(ngModel)]` directive — the square brackets bind a property to the element value (model -> view), and the parentheses listen for change events (view -> model). This is sometimes called banana-in-a-box syntax.

[illegible]

Q72. Explain Angular's dependency injection system

Dependency Injection (DI) is a design pattern where a class receives its dependencies from an external injector rather than creating them itself. Angular has a hierarchical DI system with multiple injectors (root, module, component). When you request a service in a component's constructor, Angular's injector looks up the dependency tree to find and provide the instance.

[illegible]

[illegible]

Q73. What are Angular directives? Structural vs attribute directives?

Directives are instructions in the template that tell Angular to transform the DOM. There are three types: Components (directives with a template), Structural directives (change the DOM structure — add/remove elements, prefix *), Attribute directives (change the appearance or behaviour of existing elements).

[illegible]

[illegible]

Q74. What is an Angular module (NgModule)?

An `NgModule` is a class decorated with `@NgModule` that groups related components, directives, pipes, and services. Every Angular app has at least one root module (`AppModule`). Modules define what is available in their templates via declarations, what external Angular modules they need via imports, what services they register via providers, and what they export for other modules to use.

[illegible]

Q75. What are Angular pipes? How do you create a custom pipe?

Pipes are template functions that transform displayed values. They use the `|` syntax in templates. Angular provides built-in pipes: `date`, `currency`, `decimal`, `percent`, `json`, `uppercase`, `lowercase`, `async` (for Observables/Promises). Custom pipes let you create reusable transformations.

[illegible]

Q76. Explain Subject, BehaviorSubject, and ReplaySubject in RxJS

These are special RxJS Observables that are also Observers — you can both subscribe to them AND push values into them with `.next()`. They are used extensively in Angular for sharing state between components.

Subject: No initial value. New subscribers only get future emissions. BehaviorSubject: Requires an initial value.

New subscribers immediately get the LAST emitted value. Perfect for current state (auth user, theme).

ReplaySubject: Replays the last N emissions to new subscribers. Useful for event logs.

```
import { Subject, BehaviorSubject, ReplaySubject } from 'rxjs';

// ■■ Subject ████████████████████████████████████████████
const subject = new Subject<string>();
subject.subscribe(v => console.log('A:', v));
subject.next('hello');           // A: hello
// New subscriber after the fact:
subject.subscribe(v => console.log('B:', v));
subject.next('world');          // A: world, B: world   (B missed 'hello')

// ■■ BehaviorSubject – for current state ████████████████████████████████████████████
const user$ = new BehaviorSubject<User | null>(null); // initial: null
user$.subscribe(u => console.log('user:', u));        // immediately: null

authService.login(creds).subscribe(user => user$.next(user));
// After login:
user$.subscribe(u => console.log(u?.name)); // immediately gets current user

// ■■ Angular service using BehaviorSubject ████████████████████████████████████████████
@Injectable({ providedIn: 'root' })
export class CartService {
    private items$ = new BehaviorSubject<CartItem[]>([]);
```


[illegible]

Q77. What is Change Detection in Angular? What is OnPush?

Change Detection is Angular's process of checking if the model has changed and if the view needs to be updated. By default, Angular checks every component in the tree after every async event (click, HTTP response, setTimeout). With OnPush strategy, a component only re-checks when its input references change, an event originates from the component, an Observable it subscribes to with async pipe emits, or you manually trigger it.

[illegible]

Q78. Explain Angular's router — RouterModule and lazy loading

Angular Router maps URL paths to components and manages navigation without full page reloads. It supports nested routes, route guards, route parameters, and lazy loading of feature modules (loading module JS bundles only when needed, reducing initial bundle size).

```
// app-routing.module.ts
const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'login', component: LoginComponent },

  // Lazy loaded feature module:
  {
    path: 'dashboard',
    loadChildren: () => import('./dashboard/dashboard.module')
      .then(m => m.DashboardModule),
    canActivate: [AuthGuard] // protect the route
  },

  // Lazy loaded standalone component (Angular 14+):
  {
    path: 'profile',
    loadComponent: () => import('./profile/profile.component')
      .then(m => m.ProfileComponent)
  },

  // Route with params
  { path: 'users/:id', component: UserDetailComponent },

  // Redirect and wildcard
  { path: '', redirectTo: 'home', pathMatch: 'full' },
  { path: '**', component: NotFoundComponent }
];

// ■■■ In component: navigate programmatically ■■■■■■■■■■■■■■■■■■■■■■
@Component({})
export class LoginComponent {
  constructor(private router: Router, private route: ActivatedRoute) {}

  login() {
    // Navigate to dashboard after login
    this.router.navigate(['/dashboard']);
    // Or with query params:
    this.router.navigate(['/search'], { queryParams: { q: 'Angular' } });
  }

  ngOnInit() {
    // Read route params
    this.route.params.subscribe(params => {
      const id = params['id'];
    });
    // Read query params
    this.route.queryParams.subscribe(params => {
      const page = params['page'];
    });
  }
}
```

Q79. What is the difference between Promise and Observable?

Promise: handles a single async value, eagerly executes (starts immediately on creation), not cancellable, part of JavaScript standard. Observable: handles zero or more values over time (stream), lazy (doesn't execute until subscribed), cancellable (unsubscribe), composable with powerful operators (map, filter, debounce, switchMap). Angular is heavily Observable-based via RxJS.

[illegible]

[illegible]

Q80. What are Angular Guards? (CanActivate, CanDeactivate)

Route guards are interfaces that let you control navigation. They run before Angular activates or deactivates a route. CanActivate: decides if a route can be entered (auth checks). CanDeactivate: decides if you can leave a route (unsaved changes warning). CanLoad: decides if a lazy module can be loaded.

[illegible]

[illegible]

ROUND 2 — Node.js (Q81–Q90)

Q81. What is the Node.js event loop?

Node.js uses a single-threaded, non-blocking I/O model powered by libuv. The event loop has distinct phases executed in a fixed order: timers (setTimeout/setInterval callbacks), pending callbacks (I/O callbacks deferred to next iteration), idle/prepare (internal), poll (retrieve new I/O events — blocks here if nothing else pending), check (setImmediate callbacks), close callbacks. Between each phase, Node drains both the nextTick queue and the microtask (Promise) queue.

```
// ■■ Phase execution order ██████████  
console.log('1 start');  
  
setTimeout(() => console.log('2 setTimeout'), 0);           // timers phase  
setImmediate(() => console.log('3 setImmediate'));         // check phase  
process.nextTick(() => console.log('4 nextTick'));          // nextTick queue  
Promise.resolve().then(() => console.log('5 Promise'));      // microtask queue  
  
console.log('6 end');
```

// Output:
// 1 start
// 6 end
// 4 nextTick <- nextTick drains first (before microtasks)
// 5 Promise <- microtask queue
// 2 setTimeout <- timers phase
// 3 setImmediate <- check phase

// ■■ Practical implication: never block the event loop ██████████
// BAD – synchronous file read blocks ALL requests
const data = fs.readFileSync('large-file.txt');

// GOOD – async – event loop continues while OS reads the file
fs.readFile('large-file.txt', (err, data) => {
 // process data here
});

// For CPU-heavy work: use worker threads
const { Worker } = require('worker_threads');
const worker = new Worker('./heavy-computation.js', { workerData: input });
worker.on('message', result => handleResult(result));

Q82. What is the difference between `process.nextTick()` and `setImmediate()`?

Both schedule callbacks to run asynchronously, but at different points. `process.nextTick()` executes before the event loop continues to its next phase — callbacks are queued in the `nextTick` queue which is drained after every operation, even before Promise microtasks. `setImmediate()` executes in the check phase of the event loop — after I/O events. Always prefer `setImmediate` over `nextTick` for I/O-related callbacks to avoid starving the event loop.

[illegible]

```

}

// ■■ Real use of nextTick: emit events after constructor ■■■■■■■■■■
class EventEmitter2 {
  constructor() {
    // Event handler likely attached after constructor
    // So emit on next tick to give caller time to attach handler
    process.nextTick(() => {
      this.emit('ready');
    });
  }
}

const ee = new EventEmitter2();
ee.on('ready', () => console.log('Ready!')); // attached in time

```

Q83. What are streams in Node.js?

Streams are objects that let you read or write data piece by piece (in chunks) instead of loading everything into memory at once. This is crucial for handling large files, HTTP requests/responses, or any large data sets. Types: Readable (file reading, HTTP response), Writable (file writing, HTTP request), Duplex (both read/write), Transform (duplex that transforms data, e.g., `zlib.createGzip()`).

[illegible]

Q84. Explain the cluster module in Node.js

Node.js runs on a single thread. The cluster module allows you to create child processes (workers) that share the same server port, enabling you to use all CPU cores. The master process forks workers and distributes incoming connections among them. PM2 is a popular process manager that handles clustering automatically.

[illegible]

Q85. What is middleware in Express.js?

Middleware functions are functions that have access to the request (req), response (res), and the next middleware function (next). They form a chain — each middleware can modify req/res, end the request, or call next() to pass control to the next middleware. Middleware runs in the order it is defined.

[illegible]

[illegible]

Q86. How do you handle errors in Express.js?

Express has synchronous and asynchronous error handling. Synchronous errors can be thrown directly. For async errors, you must either call `next(error)` manually or use a wrapper utility. A central error handler middleware at the end of the chain catches all errors.

```
// ■■ Async error wrapper utility ■■■
const asyncHandler = fn => (req, res, next) => {
  Promise.resolve(fn(req, res, next)).catch(next);
};

// With express 5.x, async errors auto-propagate without this

// ■■ Custom error classes ■■■
class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
    this.isOperational = true;
  }
}

class NotFoundError extends AppError {
  constructor(resource) { super(`${resource} not found`, 404); }
}

class ValidationError extends AppError {
  constructor(msg) { super(msg, 400); }
}

// ■■ Route using asyncHandler ■■■
app.get('/users/:id', asyncHandler(async (req, res) => {
  const user = await User.findById(req.params.id);
  if (!user) throw new NotFoundError('User');
  res.json(user);
})));

// ■■ Centralized error handler (must be last middleware) ■■■
app.use((err, req, res, next) => {
  let { statusCode = 500, message, isOperational } = err;

  // Log unexpected errors
  if (!isOperational) {
    console.error('UNEXPECTED ERROR:', err);
    message = 'Something went wrong';
  }

  res.status(statusCode).json({
    status: statusCode < 500 ? 'fail' : 'error',
    message: process.env.NODE_ENV === 'production' && !isOperational
      ? 'Internal server error' : message
  });
});
```


Q87. What is the difference between require and import?

require is CommonJS (CJS) — Node.js's original module system. It is synchronous, dynamic (can be called conditionally at runtime), and evaluates the module at runtime. import is ES Modules (ESM) — the JavaScript standard. It is asynchronous, static (must be at top level), evaluated at parse time (enabling tree-shaking), and supports top-level await. Modern Node.js (12+) supports both.

[illegible]

Q88. How do you handle file uploads in Node.js?

Multipart form data containing files needs special parsing. Multer is the standard Express middleware for handling multipart/form-data. For production, files are typically stored in cloud storage (AWS S3, Cloudinary) rather than the server's file system.

Q89. What is libuv in Node.js?

libuv is the C library that provides Node.js with its event loop, thread pool, and cross-platform async I/O support. It abstracts different OS async mechanisms (epoll on Linux, kqueue on macOS, IOCP on Windows) into a unified API. The thread pool (default size 4, configurable via `UV_THREADPOOL_SIZE`) handles file system operations, DNS lookups, crypto, and zlib — operations that cannot be done with non-blocking OS APIs.

```
// libuv thread pool is used transparently by these operations:

const fs      = require('fs');
const crypto  = require('crypto');
const dns     = require('dns');

// ■■ These use the thread pool (blocking in worker threads) ■■■■
fs.readFile('file.txt', cb);           // file I/O
dns.lookup('example.com', cb);         // DNS resolution
crypto.pbkdf2('password', 'salt', 100000, 64, 'sha512', cb); // CPU-intensive crypto
```

```
// ■■ These use OS non-blocking APIs (no thread pool) ■■■■■■■■■■
const net = require('net');
const server = net.createServer();    // TCP sockets – OS async
server.listen(3000);

const http = require('http');
http.get('http://example.com', cb);    // HTTP – socket-based, no thread pool

// ■■ Increasing thread pool for I/O heavy apps ■■■■■■■■■■
// In your start script or .env:
// UV_THREADPOOL_SIZE=16 node app.js

// Or programmatically (must be set before any libuv operations):
process.env.UV_THREADPOOL_SIZE = 16;

// ■■ Checking event loop utilization ■■■■■■■■■■
const { monitorEventLoopDelay } = require('perf_hooks');
const h = monitorEventLoopDelay({ resolution: 20 });
h.enable();
setTimeout(() => {
  h.disable();
  console.log('Event loop mean delay:', h.mean / 1e6, 'ms');
  console.log('Max delay:', h.max / 1e6, 'ms');
}, 5000);
```

Q90. How do you prevent callback hell in Node.js?

Callback hell (or pyramid of doom) is deeply nested callbacks that make code hard to read and error-prone.

Solutions: use Promises (then chains), async/await, modularize callbacks into named functions, use utility libraries.

```
// ■■ Callback hell (the problem) ■■■■■■■■■■
db.query('SELECT * FROM users WHERE id = ?', [userId], (err, user) => {
  if (err) return handleError(err);
  db.query('SELECT * FROM orders WHERE user_id = ?', [user.id], (err, orders) => {
    if (err) return handleError(err);
    db.query('SELECT * FROM products WHERE id = ?', [orders[0].productId], (err, product) => {
      if (err) return handleError(err);
      // 3 levels deep and growing...
      sendResponse({ user, orders, product });
    });
  });
});

// ■■ Solution 1: Promises ■■■■■■■■■■
db.queryAsync('SELECT * FROM users WHERE id = ?', [userId])
  .then(user => Promise.all([user, db.queryAsync('SELECT ...', [user.id])]))
  .then(([user, orders]) => sendResponse({ user, orders }))
  .catch(handleError);

// ■■ Solution 2: async/await (cleanest) ■■■■■■■■■■
async function getUserData(userId) {
  try {
    const user = await db.queryAsync('SELECT * FROM users WHERE id = ?', [userId]);
    const orders = await db.queryAsync('SELECT * FROM orders WHERE user_id = ?', [user.id]);
    const product = await db.queryAsync('SELECT * FROM products WHERE id = ?', [orders[0].productId]);
    return { user, orders, product };
  } catch (err) {
    handleError(err);
  }
}

// ■■ Solution 3: util.promisify (convert callback APIs) ■■■■■■■■■■
const util = require('util');
const fs = require('fs');
const readFile = util.promisify(fs.readFile);

async function readConfig() {
  const data = await readFile('./config.json', 'utf8');
  return JSON.parse(data);
}
```

```
// Or use fs.promises directly (modern Node.js):  
const data = await fs.promises.readFile('./config.json', 'utf8');
```

ROUND 2 — MongoDB (Q91–Q100)

Q91. What is a document in MongoDB?

A document is the basic unit of data in MongoDB — equivalent to a row in SQL. Documents are stored in BSON (Binary JSON) format, which extends JSON to include additional data types (ObjectId, Date, Binary, etc.).

Documents within a collection can have different fields (schemaless / dynamic schema), though Mongoose adds schema validation on the application side.

Q92. Difference between SQL and NoSQL databases

SQL databases (PostgreSQL, MySQL) store data in structured tables with predefined schemas and enforce ACID transactions. They use SQL for queries and excel at complex joins and relational data. NoSQL databases (MongoDB, Redis, Cassandra) trade rigid schemas for flexibility, horizontal scalability, and performance for specific access patterns. MongoDB is document-oriented, Redis is key-value, Cassandra is wide-column.

Q93. What is indexing in MongoDB and why is it important?

Without an index, MongoDB must scan every document in a collection (COLLSCAN) to find matching documents — $O(n)$. Indexes store a sorted subset of data (like a book's index) allowing MongoDB to jump directly to relevant documents (IXSCAN) — $O(\log n)$. Without indexes on large collections, queries can be 1000x slower. Every collection gets a default index on `_id`.

Q94. Explain the aggregation pipeline in MongoDB

The aggregation pipeline is a framework for data transformation and analysis. Documents pass through a series of stages, each transforming the data. It is MongoDB's equivalent of SQL GROUP BY, JOIN, HAVING, and window functions. Stages execute in sequence, and MongoDB optimizes the pipeline automatically.

```
// ■ Sales analytics pipeline
db.orders.aggregate([
  // Stage 1: Filter
  { $match: {
    createdAt: { $gte: new Date('2026-01-01') },
    status: 'completed'
  }},

  // Stage 2: Join with users collection
  { $lookup: {
    from: 'users',
    localField: 'userId',
    foreignField: '_id',
    as: 'user'
  }},
  { $unwind: '$user' }, // flatten the user array

  // Stage 3: Group and aggregate
  { $group: {
    _id: '$user.country',
    totalRevenue: { $sum: '$total' },
    orderCount: { $count: {} },
    avgOrder: { $avg: '$total' },
    topOrder: { $max: '$total' }
  }},

  // Stage 4: Add computed fields
  { $addFields: {
    revenuePerOrder: { $divide: ['$totalRevenue', '$orderCount'] }
  }},

  // Stage 5: Sort
  { $sort: { totalRevenue: -1 } },

  // Stage 6: Limit
  { $limit: 10 },

  // Stage 7: Shape output
  { $project: {
    country: '$_id',
    revenue: '$totalRevenue',
    orders: '$orderCount',
    avgRevenue: { $round: ['$revenuePerOrder', 2] },
    _id: 0
  } }
]);

// ■ Mongoose aggregation
const result = await Order.aggregate([
  { $match: { userId: mongoose.Types.ObjectId(userId) } },
  { $group: { _id: '$status', count: { $sum: 1 }, total: { $sum: '$amount' } } }
]);
```

Q95. What is sharding in MongoDB?

Sharding is MongoDB's horizontal scaling strategy that distributes data across multiple machines (shards). Each shard is a replica set that holds a subset of the data. A mongos router directs queries to the appropriate shard(s) based on the shard key. Config servers store the cluster metadata.

Q96. How do you handle relationships in MongoDB? (embedding vs referencing)

MongoDB offers two strategies for relationships. Embedding (denormalization): store related data inside the same document. Best for data that is always read together, rarely changes, and has limited growth. Referencing (normalization): store references (`_id`) to documents in other collections and use `$lookup` or Mongoose `populate`. Best for large sub-documents, frequently updating related data, or many-to-many relationships.

[illegible]


```
quantity:    Number,
});
```

Q97. What is Mongoose and why is it used?

Mongoose is an ODM (Object Document Mapper) for Node.js and MongoDB. It provides: schema definition with validation and type casting, built-in methods for CRUD operations, middleware (pre/post hooks), virtuals (computed properties), population (reference joining), and a Query builder API. It brings structure and safety to MongoDB's flexible schema model.

[illegible]

Q98. How do you perform transactions in MongoDB?

Multi-document ACID transactions were introduced in MongoDB 4.0 (replica sets) and 4.2 (sharded clusters). They allow multiple operations across multiple documents/collections to be atomic. Like SQL transactions, they either all succeed or all roll back. Use them for operations where consistency across multiple documents is required (e.g., transferring money, creating order + reducing inventory).

[illegible]

Q99. What is the difference between find() and findOne()?

`find()` returns a Query cursor that can match multiple documents. It returns an array (or empty array if none match). `findOne()` returns the first document that matches, or null if none match. `findOne()` is equivalent to `find().limit(1)` but is more explicit and stops after the first match.

ROUND 2 — Express.js (Q101–Q108)

Q101. What is Express.js and why is it used?

Express is a minimal, unopinionated web framework for Node.js. It provides a thin layer of web application features on top of Node's built-in HTTP module: routing, middleware, request/response helpers, and template engine support. It is the most popular Node.js framework, used directly or as the foundation for frameworks like NestJS, Feathers, and LoopBack.

[illegible]

Q102. How do you set up routing in Express.js?

Express supports inline routing (`app.get/post` etc.) and `Router` — a mini Express app for modular routing. For larger applications, use `express.Router()` to group related routes into separate files, then mount them on the main app.

[illegible]

```
app.use('/api/products', require('./routes/products'));
app.use('/api/orders', require('./routes/orders'));
app.use('/api/auth', require('./routes/auth'));

// ■■ controllers/userController.js – separation of concerns ■■■■
const UserController = {
  getAll: async (req, res, next) => {
    try {
      const users = await User.find({ isActive: true });
      res.json({ status: 'success', data: users });
    } catch (err) { next(err); }
  },
  getById: async (req, res, next) => {
    try {
      const user = await User.findById(req.params.id);
      if (!user) return res.status(404).json({ error: 'User not found' });
      res.json(user);
    } catch (err) { next(err); }
  }
};
```

Q103. What is middleware and how does the middleware chain work?

Already covered in Q85. To add more depth — middleware in Express processes requests in the order they are defined. Each middleware receives (req, res, next). Calling next() passes control to the next matching middleware or route handler. Not calling next() (without sending a response) will cause the request to hang. Calling next(error) skips all regular middleware and goes to the error handler.

[illegible]

```
}
} );
```

Q104. Difference between app.use() and app.get()

`app.use()` is for mounting middleware and sub-routers. It matches ANY HTTP method (GET, POST, PUT, etc.) and uses prefix matching — the path only needs to START with the given path. `app.get()` (and `app.post()`, `app.put()` etc.) are for specific HTTP methods and exact path matching (with params).

Q105. How do you implement authentication using JWT in Express?

JWT (JSON Web Token) authentication uses signed tokens. The client logs in, receives an access token (short-lived) and refresh token (long-lived, stored in httpOnly cookie). On each request, the client sends the access token in the Authorization header. The server verifies the signature without hitting the database. When the access token expires, the client uses the refresh token to get a new one.

Q106. What is CORS and how do you enable it in Express?

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that restricts web pages from making requests to a different origin (domain, protocol, or port) than the one that served the page. The server must send CORS headers to tell the browser which origins are allowed. Express's cors package simplifies this.

```
res.setHeader('Access-Control-Allow-Credentials', 'true');
if (req.method === 'OPTIONS') return res.sendStatus(204); // preflight
next();
});
```

Q107. How do you handle file uploads using Multer?

Already covered in Q88 (Node.js section). Additional Express-specific details: Multer integrates as Express middleware and handles the multipart/form-data parsing. The uploaded file is available on `req.file` (single) or `req.files` (multiple). Always validate file type, size, and sanitize filenames in production.

[illegible]

Q108. What is the difference between HTTP PUT and PATCH methods?

PUT is an idempotent method that replaces the ENTIRE resource at the given URL with the request body. If you PUT with only some fields, missing fields are set to null/removed. PATCH is a partial update — only the fields in

the request body are updated. PATCH is more bandwidth-efficient and safer for partial updates.

[illegible]

ROUND 3 — System Design & Architecture (Q109–Q129)

Q109. Design a URL shortener (like bit.ly) using MERN/MEAN stack

A URL shortener maps a long URL to a short code. Requirements: generate unique short codes, redirect short URLs to originals, handle high read traffic, track click analytics. Architecture: Node/Express API, MongoDB for storage, Redis cache for hot URLs, base62 encoding for codes.

```
// ■ MongoDB Schema
const urlSchema = new mongoose.Schema({
  shortCode: { type: String, unique: true, index: true },
  originalUrl: { type: String, required: true },
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  clicks: { type: Number, default: 0 },
  expiresAt: { type: Date, index: { expireAfterSeconds: 0 } },
}, { timestamps: true });
const Url = mongoose.model('Url', urlSchema);

// ■ Base62 encoding
const CHARS = '0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ';
function toBase62(num) {
  let result = '';
  while (num > 0) {
    result = CHARS[num % 62] + result;
    num = Math.floor(num / 62);
  }
  return result.padStart(6, '0');
}

// ■ Auto-incrementing counter (for unique IDs)
const counterSchema = new mongoose.Schema({ seq: Number });
async function getNextId() {
  const counter = await Counter.findByIdAndUpdate(
    'url', { $inc: { seq: 1 } }, { new: true, upsert: true }
  );
  return counter.seq;
}

// ■ Create short URL
app.post('/api/shorten', requireAuth, async (req, res) => {
  const { originalUrl, customCode, expiryDays } = req.body;

  // Validate URL
  try { new URL(originalUrl); } catch { return res.status(400).json({ error: 'Invalid URL' }); }

  let shortCode = customCode;
  if (!shortCode) {
    const id = await getNextId();
    shortCode = toBase62(id);
  }
  if (await Url.exists({ shortCode })) {
    return res.status(409).json({ error: 'Code taken' });
  }

  const url = await Url.create({
    shortCode, originalUrl, userId: req.user.id,
    expiresAt: expiryDays ? new Date(Date.now() + expiryDays * 86400000) : undefined
  });
  res.json({ shortUrl: `https://sho.rt/${shortCode}`, url });
});

// ■ Redirect (high traffic – cache with Redis)
const redis = require('redis').createClient({ url: process.env.REDIS_URL });

app.get('/:code', async (req, res) => {
  const { code } = req.params;

  // Check Redis cache first (O(1))
  const cached = await redis.get(`url:${code}`);
```

```

if (cached) {
  // Async increment (don't wait)
  Url.findOneAndUpdate({ shortCode: code }, { $inc: { clicks: 1 } }).exec();
  return res.redirect(301, cached);
}

const url = await Url.findOneAndUpdate(
  { shortCode: code },
  { $inc: { clicks: 1 } },
  { new: true }
);

if (!url) return res.status(404).send('Short URL not found');
if (url.expiresAt && url.expiresAt < new Date())
  return res.status(410).send('URL expired');

// Cache for 1 hour
await redis.setEx(`url:${code}`, 3600, url.originalUrl);
res.redirect(301, url.originalUrl);
});

```

Q110. How would you design a real-time chat application?

Real-time chat requires bidirectional communication between server and clients. Architecture: Socket.io for WebSocket connections, Redis Pub/Sub adapter to share messages across multiple Node instances, MongoDB for persistent message storage, rooms/channels for group chat, JWT for socket authentication.

[illegible]

```
// Send last 50 messages
const messages = await Message.find({ roomId })
  .sort({ createdAt: -1 }).limit(50).lean();
socket.emit('messages:history', messages.reverse());

// Notify others
socket.to(roomId).emit('user:joined', { user: socket.user, roomId });
});

// Send message
socket.on('message:send', async ({ roomId, content, type = 'text' }) => {
  const message = await Message.create({
    roomId, content, type,
    sender: { userId: socket.user.id, name: socket.user.name }
  });
  // Broadcast to everyone in room (including sender)
  io.to(roomId).emit('message:new', message);
});

// Typing indicator
socket.on('typing:start', ({ roomId }) => {
  socket.to(roomId).emit('typing:start', { user: socket.user.name, roomId });
});

socket.on('disconnect', () => {
  io.emit('user:offline', { userId: socket.user.id });
});
});
```

Q112. How do you implement pagination in a MERN app?

There are two main pagination strategies: offset/limit (page-based) and cursor-based. Offset pagination is simple but slow on large datasets (SKIP is $O(n)$). Cursor pagination uses the last document's ID to fetch the next page — $O(\log n)$ with a proper index. Use cursor for high-scale feeds, offset for admin tables.

```
// ■■■ Offset pagination ■■■
// GET /api/products?page=3&limit=20&sort=-createdAt&category=books
app.get('/api/products', async (req, res) => {
  const { page = 1, limit = 20, sort = '-createdAt', category } = req.query;
  const filters = category ? { category } : {};
  const skip = (page - 1) * limit;

  const [data, total] = await Promise.all([
    Product.find(filters).sort(sort).skip(skip).limit(Number(limit)).lean(),
    Product.countDocuments(filters)
  ]);

  res.json({
    data,
    pagination: {
      page: Number(page),
      limit: Number(limit),
      total,
      totalPages: Math.ceil(total / limit),
      hasNext: page < Math.ceil(total / limit),
      hasPrev: page > 1,
    }
  });
});

// ■■■ Cursor-based pagination (for infinite scroll / high scale) ■■■
// GET /api/feed?cursor=507f1f77bcf86cd799439011&limit=20
app.get('/api/feed', requireAuth, async (req, res) => {
  const { cursor, limit = 20 } = req.query;
  const query = { userId: { $in: req.user.following } };

  if (cursor) query._id = { $lt: mongoose.Types.ObjectId(cursor) };

  const posts = await Post.find(query)
    .sort({ _id: -1 }) // newest first - uses default _id index
```

[illegible]

Q113. Explain microservices architecture vs monolithic — when to use which?

A monolith is a single deployable unit where all modules run in the same process. Microservices splits the application into small, independent services that communicate over HTTP/gRPC/message queues. Each service owns its data and can be deployed independently.

Use monolith when: small team, new product (move fast), simple domain. Use microservices when: large team (Conway's Law), need to scale specific parts independently, different services need different tech stacks, or you need fault isolation.

```
// ■■■ MONOLITH structure ████████████████████████████████████████████  
// src/  
//   routes/      users.js, orders.js, products.js  
//   models/       User.js, Order.js, Product.js  
//   services/    emailService.js, paymentService.js  
//   app.js        - single entry point  
  
// ■■ MICROSERVICES structure ████████████████████████████████████████████  
// services/  
//   user-service/     port 3001  (users DB)  
//   order-service/    port 3002  (orders DB)  
//   product-service/  port 3003  (products DB)  
//   notification-service/ port 3004  
//   api-gateway/      port 3000  (single entry point for clients)  
  
// ■■ API Gateway pattern ████████████████████████████████████████████  
const gateway = express();  
const httpProxy = require('http-proxy-middleware');  
  
gateway.use('/api/users',      httpProxy.createProxyMiddleware({ target: 'http://user-service:3001' }));  
gateway.use('/api/orders',    httpProxy.createProxyMiddleware({ target: 'http://order-service:3002' }));  
gateway.use('/api/products',  httpProxy.createProxyMiddleware({ target: 'http://product-service:3003' }));
```

```
// ■■■ Inter-service communication with message queue ■■■  
// Order service publishes event when order placed:  
const amqp = require('amqplib');  
  
async function publishOrderPlaced(order) {  
  const conn = await amqp.connect(process.env.RABBITMQ_URL);  
  const channel = await conn.createChannel();  
  await channel.assertExchange('orders', 'topic', { durable: true });  
  channel.publish('orders', 'order.placed', Buffer.from(JSON.stringify(order)));  
}  
  
// Notification service subscribes to the event:  
async function subscribeToOrders() {  
  const channel = await getChannel();  
  await channel.assertQueue('notifications.order-placed', { durable: true });  
  await channel.bindQueue('notifications.order-placed', 'orders', 'order.placed');  
  channel.consume('notifications.order-placed', async (msg) => {  
    const order = JSON.parse(msg.content);  
    await sendOrderConfirmationEmail(order);  
    channel.ack(msg);  
  });  
}
```

Q115. How would you implement rate limiting in a Node.js API?

Rate limiting protects your API from abuse, DDoS attacks, and ensures fair usage. For single-server setups, `express-rate-limit` with in-memory store is sufficient. For multi-server (clustered or microservices), use Redis as the shared store so rate limit counters are consistent across all instances.

[illegible]

```
const data = await redis.hGetAll(key);
let tokens = parseFloat(data.tokens || capacity);
let lastRefill = parseFloat(data.lastRefill || now);

// Refill tokens
const elapsed = (now - lastRefill) / 1000;
tokens = Math.min(capacity, tokens + elapsed * refillRate);

if (tokens < 1) return false; // rate limited

tokens -= 1;
await redis.hSet(key, { tokens: tokens.toString(), lastRefill: now.toString() });
await redis.expire(key, 3600);
return true; // request allowed
}
```

Q124. What is Redis and how do you use caching in a Node.js app?

Redis is an in-memory data structure store used as a cache, message broker, and queue. Its sub-millisecond latency makes it ideal for caching expensive database queries, session storage, rate limiting, and pub/sub. A cache-aside strategy: check Redis first, if miss hit the DB, store result in Redis with TTL.

[illegible]

```
app.use(session({
  store: new RedisStore({ client }),
  secret: process.env.SESSION_SECRET,
  resave: false, saveUninitialized: false,
  cookie: { secure: true, maxAge: 24 * 60 * 60 * 1000 }
}));
```

Q125. How do you handle WebSockets with Node.js (Socket.io)?

Already covered in Q110 with detailed chat example. Additional Socket.io patterns: namespaces (logical separation within one server), rooms (dynamic groups), acknowledgements (confirmed delivery), and broadcasting strategies.

[illegible]

Q127. What is GraphQL and how does it compare to REST?

GraphQL is a query language for APIs and a server-side runtime. Instead of multiple REST endpoints, you have a single `/graphql` endpoint where clients specify exactly what data they need in their query. This eliminates over-fetching (getting too much data) and under-fetching (needing multiple requests). It has a strongly typed schema that serves as a contract between client and server.

Q129. How do you monitor and log production Node.js applications?

Production monitoring combines structured logging (Winston), error tracking (Sentry), performance monitoring (APM tools), and health check endpoints. The goal is to detect issues before users do and have enough context to diagnose them quickly.

ROUND 4 — Manager / Delivery Head Round (Q130–Q148)

Q130. How did you deploy your project to the cloud? (AWS ECS + CI/CD)

A typical MERN production deployment uses AWS ECS (Fargate) for containerized Node.js API, S3 + CloudFront for the React SPA, MongoDB Atlas for managed database, and GitHub Actions for CI/CD. The pipeline builds Docker images, runs tests, and deploys on merge to main.

```
--cluster production \  
--service api-service \  
--force-new-deployment
```

Q133. How do you ensure code quality in your projects?

Code quality is enforced through automated tooling (linting, formatting, testing) and process (code reviews, PR checks). The goal is to catch issues early and keep the codebase consistent, regardless of who wrote the code.

Q134. What is your approach to writing unit tests? (Jest, Mocha, Chai)

Unit testing verifies individual functions/modules in isolation. The AAA pattern (Arrange, Act, Assert) structures each test clearly. Mock external dependencies (databases, APIs, email) to keep tests fast and deterministic. Integration tests test the full HTTP layer with a real database (using a test DB or MongoDB memory server).

Q135. How do you handle technical debt?

Technical debt is code that is quick to write but costly to maintain. It accumulates when teams take shortcuts under time pressure. The key is to make it visible, track it, and allocate regular time to address it — rather than ignoring it until it becomes a crisis.

Strategies: 1) Track debt in your issue tracker with a 'tech-debt' label. 2) Dedicate 20% of each sprint to debt reduction. 3) Use the Boy Scout Rule — leave code better than you found it. 4) Set quality gates (coverage thresholds, lint rules) to prevent new debt. 5) Refactor incrementally with tests as a safety net — never rewrite everything at once.

ROUND 5 — HR Round (Q149–Q153)

Q149. What are your salary expectations?

Research market rates for your role, experience level, and location on Glassdoor, LinkedIn Salary, and Levels.fyi. For mid-level MERN/MEAN developers in India (2026), the range is approximately ■12–25 LPA depending on company and city. For senior roles, ■25–45 LPA. Anchor your expectations 15–20% above your actual minimum.

Sample answer: 'Based on my research for this role and my [X] years of experience with the MERN stack — including [specific skills: microservices, AWS, etc.] — I'm looking at [range]. That said, I'm flexible based on the total compensation package including benefits, learning opportunities, and growth potential. What is the budgeted range for this position?'

Tips: Always give a range, not a single number. Ask for their range to let them anchor first. Factor in the full package (equity, bonuses, WFH flexibility). Never share your current salary unless legally required.

Q151. How soon can you join?

Be honest about your notice period. Most Indian IT companies have a 30–90 day notice period. Negotiate if possible. If you can join sooner (buyout option), mention it as a possibility, not a guarantee.

Sample answer: 'My current notice period is 60 days. I'll start the relieving process as soon as I receive the offer letter. Depending on project handover, I may be able to buy out a portion of my notice — my employer permits this. I'd target joining within [45-60] days. Would that timeline work for this role?'

If you're a fresher or currently unemployed: 'I can join immediately / within 2 weeks, which gives me time to relocate and complete any joining formalities.'

Q153. What motivates you as a developer?

Be genuine. Interviewers can spot a generic answer. Connect your motivation to something specific about the role or company. Common authentic motivations: solving real problems, learning new technologies, seeing users use what you built, code quality craftsmanship, mentoring others.

Sample answer: 'I'm most motivated when I can see the direct impact of my work on real users. In my last project, I rebuilt the search feature using Elasticsearch and MongoDB text indexes — query time dropped from 2 seconds to 80ms. Watching that improvement in production and reading user feedback was incredibly satisfying. At Accion Labs, I'm excited about the opportunity to work on scalable enterprise products where my architectural decisions affect thousands of users. I also enjoy the technical depth of the MERN ecosystem — the JavaScript all-the-way-down approach means I can contribute across the stack and own features end-to-end, which keeps me engaged.'

BONUS — Advanced / Frequently Missed Topics (Q154–Q163)

Q154. What is the difference between authentication and authorization?

Authentication (AuthN) is the process of verifying WHO you are — proving your identity. 'Are you really Alice?'

Authorization (AuthZ) is determining what you are ALLOWED to do — checking your permissions after identity is confirmed. 'Alice is authenticated, but can she access /admin?'

Q155. Explain SOLID principles with JavaScript examples

SOLID is a set of five object-oriented design principles that help create maintainable, extensible, and testable code. Originally formulated by Robert C. Martin for class-based OOP, they apply equally well to JavaScript modules and functions.

Q156. What is tree-shaking in webpack?

Tree-shaking is a dead-code elimination technique used by bundlers (webpack, Rollup, Vite) to remove unused exports from the final bundle. It works by statically analysing ES Module import/export statements (which are static and analysable at build time — unlike CommonJS require). The name comes from 'shaking the tree and watching dead leaves fall'.

Q157. What is the difference between localStorage, sessionStorage, and cookies?

All three store data in the browser, but differ in capacity, persistence, scope, and server accessibility. `localStorage` persists until explicitly cleared. `sessionStorage` clears when the browser tab is closed. Cookies are sent to the server on every request and can be set server-side with `HttpOnly` for security.

Q158. What are Web Workers?

Web Workers run JavaScript in a background thread, separate from the main UI thread. Since JavaScript is single-threaded, heavy computations block rendering and make the UI unresponsive. Web Workers solve this by offloading CPU-intensive work. Workers cannot access the DOM — they communicate with the main thread via message passing.

```
useEffect(() => {
  if (rawData) workerRef.current.postMessage(rawData);
}, [rawData]);

if (!processed) return <Spinner />;
return <table>{/* render processed */}</table>;
}

// ■■ SharedArrayBuffer for shared memory between workers ■■■■■■■■
const sharedBuffer = new SharedArrayBuffer(1024);
const sharedArray = new Int32Array(sharedBuffer);
worker.postMessage({ buffer: sharedBuffer });
// Worker can read/write to same memory – use Atomics for thread safety
Atomics.add(sharedArray, 0, 1); // atomic increment
```

Q159. How does HTTPS work? (TLS handshake)

HTTPS is HTTP over TLS (Transport Layer Security). TLS provides: encryption (data can't be read in transit), authentication (server's identity is verified via certificate), integrity (data can't be tampered with). The TLS handshake establishes a secure session before any HTTP data is sent.

TLS 1.3 Handshake (simplified): 1) Client Hello: client sends supported TLS versions, cipher suites, and a random value. 2) Server Hello: server responds with chosen cipher suite, its digital certificate, and its public key. 3) Client verifies the certificate against trusted CAs. 4) Key exchange (ECDHE): both sides generate a shared session key without transmitting it. 5) Both sides send Finished message encrypted with the session key. 6) Encrypted HTTP communication begins.

```
// The load balancer handles TLS – Node.js only deals with HTTP inside the VPC
```

Q160. What is the same-origin policy?

The same-origin policy is a browser security rule that prevents a web page from making requests to a different origin than the one that served it. Origin = protocol + domain + port. Two URLs have the same origin only if ALL THREE match. This prevents malicious sites from reading your bank's data using your session cookies.

Q161. Explain CSP (Content Security Policy)

Content Security Policy is an HTTP header that tells the browser which sources of content (scripts, styles, images, fonts, etc.) are trusted. It is the primary defence against XSS (Cross-Site Scripting) attacks — even if an attacker injects a script tag, the browser won't execute it if the source isn't whitelisted.

Q162. What is the difference between npm, yarn, and pnpm?

All three are JavaScript package managers that install and manage dependencies from the npm registry. They differ in performance, disk usage, and how they handle the node modules structure.

Q163. What are monorepos and tools like Nx/Turborepo?

A monorepo is a single repository containing multiple related projects (apps, packages, services). Benefits: code sharing between projects, atomic commits across projects, single CI/CD pipeline, consistent tooling. The challenge is build performance — you don't want to rebuild everything when you change one package. Nx and Turborepo solve this with dependency graph analysis and intelligent caching.

Quick Reference Summary

Key Concepts at a Glance

JavaScript: Closures remember outer scope. Hoisting: `var`=undefined, `let/const`=TDZ. Event loop: microtasks before macrotasks. Arrow functions: lexical 'this'. Promises: single async value; Observables: stream.

React: VDOM diffs before touching real DOM. Fiber makes reconciliation interruptible. Hooks replace class lifecycle methods. `useMemo` = memoize value, `useCallback` = memoize function, `useRef` = mutable without re-render.

Angular: Two-way binding via `[(ngModel)]`. DI system injects services. `OnPush` skips change detection unless input ref changes. RxJS Observables are lazy and cancellable. `BehaviorSubject` = current state.

Node.js: Single thread + event loop + libuv thread pool. Never block the event loop with sync operations. Use `async/await`, streams for large data, `cluster/PM2` for multi-core utilization.

MongoDB: Documents are BSON objects. `COLLSCAN` = no index (slow). ESR rule for compound indexes. Aggregation pipeline for analytics. Transactions for multi-document atomicity.

Security: Authentication = who you are. Authorization = what you can do. JWT in `httpOnly` cookies. CSP prevents XSS. CORS controls cross-origin requests. Helmet adds security headers.

Performance: Redis cache = $O(1)$ vs DB = $O(\log n)$. Code splitting + lazy loading = smaller bundles. Cursor pagination > offset pagination at scale. EXPLAIN queries to check index usage.

Good luck at Accion Labs! — MERN/MEAN Interview Guide 2026